

Applicazione del Reinforcement Learning per la Navigazione Autonoma di Veicoli utilizzando Unity e ML-Agents

Gabriele Bandino

Abstract

Questo studio presenta lo sviluppo di un simulatore di guida autonoma implementato con il motore di gioco Unity e il toolkit di apprendimento automatico ML-Agents. L'obiettivo principale del progetto è esplorare le capacità di un agente macchina, allenato attraverso tecniche di reinforcement learning, di percorrere vari percorsi in maniera autonoma all'interno di un ambiente virtuale. Il simulatore si concentra sulle funzionalità base di guida, come il riconoscimento del percorso, la gestione della velocità e l'evitamento di ostacoli, senza entrare nei dettagli di funzionalità avanzate come la rilevazione di segnali stradali o la gestione del traffico. Questa limitazione è stata intenzionale per mantenere il focus sulla validazione dell'efficacia dell'apprendimento per rinforzo in un contesto controllato. I risultati mostrano che l'agente addestrato è in grado di navigare con successo attraverso diversi percorsi, dimostrando la fattibilità dell'approccio. Il progetto si pone come un punto di partenza per ulteriori indagini nel campo della simulazione di guida autonoma, sottolineando l'importanza dell'apprendimento per rinforzo come strumento efficace nell'addestramento di agenti autonomi.

1. Introduzione

Nel panorama in rapida evoluzione della tecnologia dei trasporti, le auto a guida autonoma sono emerse come un'innovazione fondamentale, promettendo di rivoluzionare il modo in cui percepiamo e interagiamo con la mobilità. L'avvento dei veicoli autonomi non rappresenta solo un

salto tecnologico, ma anche un cambiamento di paradigma nell'ingegneria automobilistica, nella pianificazione urbana e persino in un ampio spettro di norme sociali. Questa trasformazione è sostenuta dai progressi nell'apprendimento automatico e nell'intelligenza artificiale, che sono le pietre angolari dello sviluppo dei veicoli autonomi.

L'importanza delle auto a guida autonoma nel mondo di oggi può essere attribuita a diversi fattori chiave. In primo luogo, offrono il potenziale per ridurre significativamente gli incidenti stradali, la maggior parte dei quali sono causati da errori umani.

Inoltre, i veicoli autonomi sono pronti a portare notevoli miglioramenti nell'efficienza del traffico e ridurre la congestione. Ottimizzando i modelli di guida e riducendo la necessità di spazi di parcheggio, i veicoli a guida autonoma possono contribuire a una pianificazione urbana più efficiente e a una riduzione delle emissioni di carbonio.

Inoltre, le auto a guida autonoma promettono un aumento dell'accessibilità e della mobilità per un'ampia gamma della popolazione, inclusi anziani e disabili, che altrimenti potrebbero essere incapaci di guidare. Questo aspetto può portare a una società più inclusiva in cui il trasporto non è una barriera, ma un facilitatore di opportunità. [\[1.1\]](#)

La ricerca e lo sviluppo in corso in questo campo, in particolare nell'apprendimento automatico e nell'IA, non solo stanno preparando il terreno per un trasporto più sicuro ed efficiente, ma stanno anche preparando il palcoscenico per un futuro in

cui la mobilità autonoma è una parte integrante della vita quotidiana.

Un aspetto cruciale del processo di addestramento è l'utilizzo di simulazioni, un passaggio fondamentale prima di procedere ai test dal vivo. Questi simulatori, spesso dotati di avanzate tecnologie e un grande numero di dettagli, permettono agli sviluppatori di modellare una vasta gamma di scenari stradali, dalle condizioni meteorologiche avverse al comportamento imprevedibile degli altri utenti della strada. In questi ambienti virtuali, le auto possono imparare e perfezionare le loro capacità di navigazione, riconoscimento e decisione senza i rischi associati alla guida reale. Una volta che un veicolo dimostra un livello adeguato di competenza e affidabilità in simulazione, viene poi testato in condizioni reali, dove deve affrontare sfide e variabili non replicabili completamente in un ambiente virtuale. Questo passaggio dal virtuale al reale è essenziale per garantire che le auto a guida autonoma siano non solo intelligenti e capaci, ma anche sicure per le strade pubbliche. [\[1.2\]](#)

Attualmente il reinforcement learning è l'approccio dominante per quanto riguarda l'ambito della guida autonoma. L'addestramento per l'utilizzo in strada richiede ingenti risorse computazionali, nonché lunghi tempi d'attesa, per questo è ampiamente in fase di sviluppo una nuova branca, ovvero il quantum reinforcement learning.

Algoritmi come Nav-Q, superano il reinforcement learning tradizionale in stabilità e velocità di convergenza per la navigazione senza collisioni delle auto autonome. [\[1.3\]](#)

Questo studio mostra un esempio di apprendimento di guida autonoma all'interno di un ambiente simulato, dedicando attenzione alle parti fondamentali della guida, ovvero il controllo della sterzata e della velocità.

2. Strumenti e metodi

2.1 Strumenti

I seguenti strumenti e metodi hanno formato la base tecnologica e metodologica del progetto, permettendo lo sviluppo di un agente di guida autonoma sofisticato e capace di apprendere in modo efficace all'interno di un ambiente simulato.

- **Unity:** Utilizzato come motore grafico principale, Unity ha fornito una piattaforma robusta e versatile per la creazione della simulazione. Questo ambiente ha permesso la modellazione di percorsi dinamici, essenziali per testare le capacità dell'agente in scenari variabili.
- **C#:** Il linguaggio di programmazione C# è stato impiegato per sviluppare script personalizzati all'interno dell'ambiente Unity. Questi script hanno permesso l'interazione con l'ambiente simulato e la generazione di elementi dinamici, come il percorso e il sistema di checkpoint
- **Torch:** è un framework di deep learning, utilizzato nel progetto per l'addestramento dell'agente. Questo package, di cui si serve il toolkit ML-Agents, ha fornito le funzionalità necessarie per implementare e ottimizzare algoritmi di apprendimento complessi.
- **ML-Agents:** un toolkit di Unity che facilita l'addestramento di agenti intelligenti tramite tecniche di apprendimento automatico. Utilizzando Torch come package per il training, ML-Agents ha permesso di addestrare l'agente in un ambiente di simulazione, sfruttando le potenzialità di Unity.
- **TensorBoard:** utilizzato per monitorare e analizzare i risultati dell'addestramento. Questo strumento ha permesso di visualizzare metriche come la loss e il reward durante l'addestramento, offrendo una visione chiara dei progressi e delle prestazioni dell'agente.

2.2 Il Reinforcement Learning

L'intero training dell'agente è basato su una tecnica di deep learning detta Reinforcement Learning, in cui l'agente impara a svolgere azioni ottimali in base ad un sistema artificiale di reward e punizioni. Questo approccio è stato scelto per la sua efficacia nell'addestrare agenti a eseguire compiti complessi in ambienti dinamici, come la guida autonoma, dove l'agente deve imparare a navigare e reagire a situazioni imprevedibili basandosi sull'esperienza acquisita.

Possiamo semplificare il funzionamento del Reinforcement learning come:

$$R_t = \sum_{t=0}^{\infty} \gamma^t r_t$$

In cui:

- R_t è il reward totale, dato dalla sommatoria dei vari reward scontati all'istante t ;

- r_t è il reward ottenuto all'istante t ;

- γ^t è il fattore di sconto;

Il funzionamento del reinforcement learning vede una **policy** che l'agente continua ad aggiornare per arrivare ad ottenere sempre la ricompensa migliore compiendo un'azione a partire da un determinato stato.

$$\pi(a|s) = P[A|s]$$

In cui:

- π è la policy

- a sono le azioni possibili

- s è lo stato in un determinato istante t

- $P[A|s]$ è la distribuzione di probabilità sulle azioni generate

La policy dovrà inoltre gestire le azioni di *exploitation* o *exploration*.

Parliamo di **exploitation** quando l'agente si serve delle nozioni a lui già conosciute per determinare la prossima azione, mentre parliamo di **exploration** quando l'agente decide di "intraprendere un nuovo percorso" con la speranza di ottenere una ricompensa maggiore.

3. Implementazione

3.1 L'ambiente

L'ambiente di simulazione è composto da un tracciato e dei checkpoint generati tramite la funzione *RoadGen*.

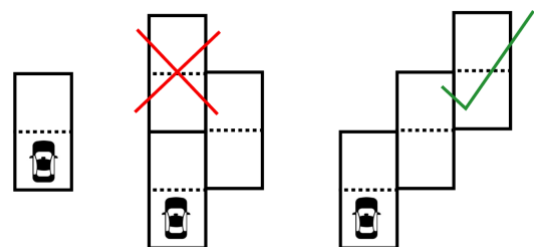
Questa funzione dispone di due variabili pubbliche, una per la selezione del prefab con cui costruire la strada, e una per la lunghezza del percorso.

I prefab che vengono inseriti all'interno del generatore devono avere larghezza 10 e profondità 20. La scelta delle dimensioni non è casuale, il prefab è un rettangolo composto da due quadrati, questo rende più e semplice e rapida la creazione del tracciato.

Partendo dal primo rettangolo che viene posizionato sotto la macchina all'inizio dell'episodio, i prossimi verranno generati secondo la seguente logica:

- Ogni rettangolo successivo può essere generato secondo 3 movimenti, uno frontale, uno diagonale a sinistra e uno diagonale a destra.
- Se il rettangolo precedente è stato generato frontalmente, il successivo sarà scelto casualmente tra tutte e tre le direzioni
- Se il rettangolo precedente è stato generato a destra, il successivo non potrà essere generato a sinistra e verrà scelto casualmente tra i restanti 2
- Se il rettangolo precedente è stato generato a sinistra, il successivo non potrà essere generato a destra e verrà scelto casualmente tra i restanti 2

Questo perché la generazione di un rettangolo prima a destra, poi a sinistra o viceversa formerebbe una strada unica, rendendo poco efficiente una parte del percorso ai fini di addestramento.



Al centro di ogni rettangolo viene poi generato un checkpoint che viene utilizzato per indirizzare la macchina lungo il percorso e tenere sotto controllo i progressi.

Ogni nuovo episodio, il tracciato viene rigenerato.
Gli episodi hanno una durata massima, modificabile dall'apposita variabile esposta sull'agente.

3.2 L'agente

Ciascun agente ha un gameObject root a cui vengono associati tutti gli script di training, generazione e controllo. Tutti i gameObject sotto la root directory sono i componenti fisici che compongono la macchina.

- **CenterOfMass:** gameObject utilizzato per tracciare la posizione centrale della macchina
- **Colliders:** due gameObject associati con un Box Collider per tracciare le collisioni del corpo della macchina.
- **WheelColliders:** collider delle ruote, ciascun gameObject ha associato ad esso un Wheel Collider.

I parametri dei Wheel Collider sono stati accuratamente selezionati e testati per rendere la guida del veicolo adeguata. Alcuni parametri di cui tener conto particolarmente sono:

- **Radius:** indica la grandezza del raggio della ruota
- **Extremum slip:** troviamo due variabili di questo tipo all'interno di ciascun Wheel Collider, vengono utilizzate per gestire l'attrito della ruota sul suolo
- **Car:** contiene tutti i gameObject che compongono la parte estetica della macchina, come ruote, corpo, luci, etc

Il modello della macchina e lo script per il controllo sono importati dal package *Low Poly Playable Vehicles*. [\[3.1\]](#)

Tuttavia, lo script di controllo della macchina *LLPV_CarController* è stato notevolmente modificato per migliorarne la funzione e renderlo adatto all'utilizzo per l'addestramento dell'agente.

All'interno dello script troviamo:

- *Max Torque, Max Brake Torque, Max Steer Angle, Down Force e Top*

Speed, ovvero diverse **variabili pubbliche** che permettono di controllare facilmente il comportamento e la velocità della macchina.

- **VisualizeWheel()** funzione per collegare la rotazione del collider delle ruote con rispettivo modello 3D.
- **Move()** funzione che riceve in input i comandi dell'utente e gestisce i movimenti della macchina. Viene utilizzata nello script *AgentCar*, pertanto la sua funzione non è quella di ricevere gli input da tastiera, ma unicamente di gestirli. Permette alla macchina di accelerare, decelerare, sterzare e tirare il freno a mano.
- **ManageSpeed()** funzione per gestire la velocità massima, valore modificabile con l'apposita variabile pubblica

4. L'addestramento

Durante l'addestramento l'agente impara a svolgere il suo compito all'interno dell'ambiente, basandosi su *rewards* e *osservazioni*, accuratamente selezionati durante la fase di creazione degli elementi per il training. L'agente attraverso vari tentativi arriva attraverso i reward alla policy ottimale.

In fase di training, all'interno dell'ambiente sono state disposte più macchine per diminuire i tempi di attesa. Per questo motivo la gestione dei checkpoint e della generazione del tracciato avviene localmente rispetto ad ogni singolo gameObject agente all'interno dell'environnement.

Gli obiettivi finali per ritenerci soddisfatti dell'addestramento sono quelli di ottenere un agente che non solo fosse capace di percorrere il percorso, ma di farlo nel minor tempo possibile.

4.1 Iperparametri

Gli iperparametri nel reinforcement learning sono parametri configurabili che influenzano il processo di apprendimento dell'agente.

Questi parametri devono essere impostati prima dell'inizio dell'apprendimento e non vengono modificati automaticamente dall'algoritmo di apprendimento durante l'addestramento.

La scelta adeguata degli iperparametri è cruciale per ottenere buone prestazioni dall'agente di apprendimento per rinforzo. Di seguito, alcuni degli iperparametri che generalmente consideriamo più importanti:

- **Learning Rate** (0,0003): Questo controlla quanto le conoscenze dell'agente vengono aggiornate con ogni nuovo pezzo di informazione. Un tasso di apprendimento più alto significa che l'agente aggiorna le sue conoscenze più aggressivamente. Un tasso di apprendimento di 0,0003 è relativamente basso, indicando aggiornamenti lenti e cauti al modello.
- **Beta** (0,005): è spesso associato al bonus di entropia nei metodi di gradiente di politica, regolando l'equilibrio tra exploration e exploitation. Un beta di 0,005 suggerisce un'enfasi moderata sull'incoraggiamento all'exploration per prevenire la convergenza prematura su politiche subottimali.
- **Epsilon** (0,2): Nel contesto delle strategie epsilon-avidità, epsilon rappresenta la probabilità di scegliere un'azione casuale rispetto alla migliore azione secondo il modello. Un epsilon di 0,2 significa che c'è una probabilità del 20% di selezione casuale dell'azione, promuovendo l'esplorazione.
- **Lambda** (λ) (0,95): Lambda è utilizzato in algoritmi come TD(λ) per combinare informazioni da più passi temporali, bilanciando efficacemente il trade-off tra bias e varianza nell'assegnazione del credito. Un lambda di 0,95 tende all'uso di sequenze di eventi più lunghe per aggiornare i valori, suggerendo una preferenza per un

processo di apprendimento più stabile, ma potenzialmente più lento.

- **Gamma** (γ) (0,98): Gamma è il fattore di sconto, che determina il valore presente delle ricompense future. Un gamma di 0,98 significa che le ricompense future sono molto valutate, ma leggermente meno delle ricompense immediate. Questo incoraggia strategie che sono vantaggiose nel lungo termine ma danno ancora priorità ai guadagni più immediati.

4.2 Rewards

Tramite i rewards, positivi o negativi, riusciamo a comunicare all'agente se stia svolgendo un buon lavoro o meno.

Inizialmente il focus è stato quello di insegnare all'agente come percorrere interamente il percorso.

Viene così assegnato un reward positivo ogni volta che il percorso viene completato, e uno negativo ogni volta che l'agente cade oltre il ciglio della strada.

La risposta dell'agente a questa combinazione è stata semplicemente stare fermo o muoversi per la pista evitando di cadere in modo da non ricevere né un reward positivo, né uno negativo.

Da ciò si può dedurre che l'agente ha bisogno di uno stimolo per proseguire lungo il tracciato.

È stato così introdotto un reward basato sulla distanza relativa tra la macchina e la fine del tracciato.

$$r_t = -\frac{|\Delta_0 - \Delta_t|}{\lambda}$$

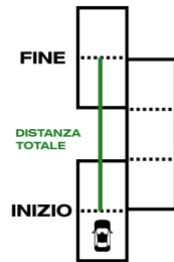
In cui:

- Δ_0 è la distanza iniziale tra la macchina e la fine del percorso;
- Δ_t è la distanza nell'istante attuale
- λ è il valore di scala, ovvero il valore che permette di scalare il reward e normalizzarlo in modo che sia adeguatamente significativo per il modello

Il reward sarà sempre negativo, in particolare all'inizio dell'episodio, in cui

l'agente punterà immediatamente a procedere nel tracciato e accorciare la sua distanza dal traguardo. Questa scelta è dovuta al fatto che, se parte di questo reward fosse stato positivo, dopo una certa distanza la macchina si sarebbe fermata a causa delle ricompense costanti.

La distanza viene misurata sul solo asse z, questo perché il tracciato generato è a sua volta sviluppato seguendo l'asse in questione.



Generalizzazione: in caso di circuito non sviluppato lungo un unico asse, potremmo calcolare la distanza totale (iniziale tra macchina e traguardo) come la sommatoria della distanza tra ogni checkpoint consecutivo



Successivamente all'inserimento di questo nuovo reward l'agente prosegue, ma fatica a terminare il percorso. Dato il limite di tempo per ogni episodio, si pensava che l'agente avrebbe aumentato la sua velocità per terminare più velocemente il percorso, così non è stato.

È stato dunque inserito un reward per fornire una ricompensa alla macchina in base alla sua velocità, e una penalizzazione nel caso in cui questa proceda lentamente.

$$r_t = \left(\frac{v_t}{v_{max}} - \delta \right) \lambda$$

In cui:

- v_t velocità all'istante t
- v_{max} velocità massima
- δ variabile di scarto, valore compreso tra 0 e 1 che permette di

definire la soglia entro la quale il reward è positivo o negativo

- λ è il valore di scala

L'agente riceve inoltre una ricompensa x ogni volta che completa il percorso, è una penalità $-x$ ogni volta che supera il ciglio della strada.

4.3 Osservazioni

Le osservazioni sono i dati che l'agente riceve in input. È importante fornire all'agente tutti i dati che porterebbero ad un risultato migliore, questo senza esagerare: l'incremento dei dati in input porta anche a un training più lungo e un modello più complesso.

Nel nostro caso, infatti, le osservazioni sono state aggiunte man mano, in base alle necessità dell'agente.

Un particolare tipo di osservazione che implementa il toolkit ML-Agents sono i **Ray Perception Sensor 3d**, che chiameremo **RPS**, ovvero dei sensori collegabili all'agente che permettono di identificare i gameObject rilevati. È possibile inserire filtri e variare il numero di "raggi/sensori", la loro angolazione rispetto al suolo e la loro distanza l'uno dall'altro.

4.3.1 Fase 1: l'inizio

Inizialmente le osservazioni comprendevano:

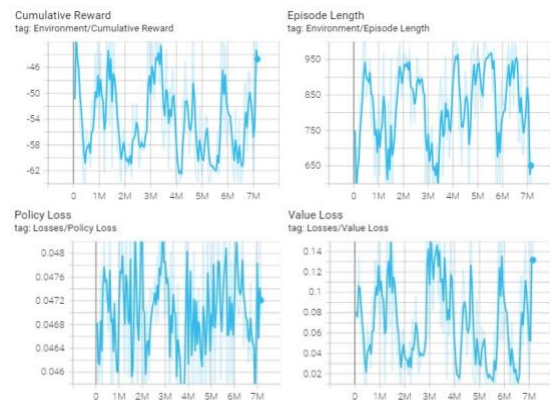
- **Posizione** della macchina
- **Velocità** della macchina
- **RPS 360sensor**, identifica la strada, a media/bassa distanza dalla macchina, ricopre a 360° la visuale dell'agente

Tensor size : 22

Durante la fase di addestramento si è potuto osservare come la macchina non riuscisse a trovare la direzione corretta o a rallentare per tempo per affrontare una curva.

Questo avveniva perché, nonostante il RPS fosse utile inizialmente per trovare la direzione in cui dirigersi, una volta acquisita velocità, la distanza di rilevazione era troppo bassa per prendere decisioni in tempo.

Inoltre, essendo il percorso da percorrere sempre diverso, l'utilizzo della posizione come osservazione era altamente irrilevante, per questo è stata rimossa nella fase successiva.



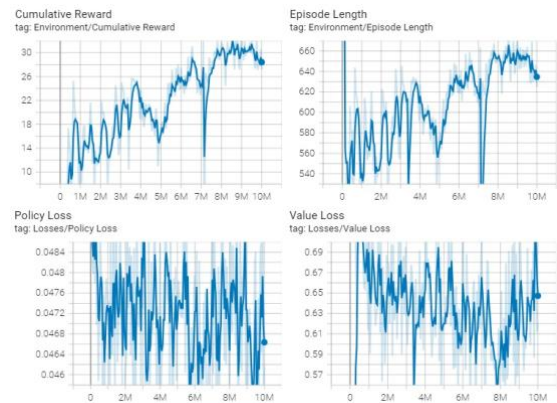
4.3.2 Fase 2: sensori posizione

Si è dunque pensato di inserire una serie di RPS per l'orientamento della macchina in velocità:

- **Velocità** della macchina;
- **RPS 360sensor**;
- **RPS 360sensor2** più vicino al suolo, utile per velocizzare la parte iniziale dell'apprendimento, inutile una volta che l'agente aumenta la sua velocità
- **RPS frontsensor1**, diretto verso il suolo, a media distanza dalla macchina, ricopre i 180° frontali;
- **RPS frontsensor2**, diretto verso il suolo, a media-lunga distanza dalla macchina, ricopre i 180° frontali;
- **RPS frontsensor3**, diretto verso il suolo, a lunga distanza dalla macchina, ricopre i 180° frontali;

Grazie a questo set di osservazioni, l'agente inizia a vedere progressi, riuscendo a percorrere tratti più lunghi di strada, fallendo però in tratti più complessi con molte "curve a S".

La macchina, infatti, spesso non riusciva a trovare una traiettoria ideale per affrontare in maniera adeguata il tracciato. Per questo sono stati introdotti i checkpoint.



4.3.3 Fase 3: i checkpoint

Il sistema di checkpoint vede una lista ordinata di pannelli identificabili dai RPS, che non sono collider, la macchina riesce così a identificarli e passare attraverso.

I 3 checkpoint prossimi rispetto all'agente sono segnati con un tag l'uno diverso dall'altro, in modo da poterli identificare singolarmente, questo perché alla lista delle osservazioni si aggiungono tre RPS dedicati al rilevamento dei checkpoint. Verranno dunque identificati il 1°, 2° e 3° checkpoint successivo e verranno inserite 3 osservazioni che permettono di calcolare l'angolo tra la macchina e il centro del checkpoint rilevato.

Il calcolo dell'angolo viene effettuato utilizzando l'operazione di Dot Product tra 2 Vector3

$$\alpha = \text{Dot}(\text{norm}(af), \text{norm}(cp - ap))$$

Dove:

- α è l'angolo calcolato
- **norm** è la funzione di normalizzazione
- **af** è *transform.forward*, ovvero il Vector3 che indica la direzione frontale alla macchina
- **cp - ap** differenza tra *checkpointdir.position* e *transform.position*, ovvero la differenza tra la posizione del checkpoint e della macchina

Conoscendo l'angolazione per dirigersi verso il checkpoint, l'agente riesce a predire in maniera più adeguata la

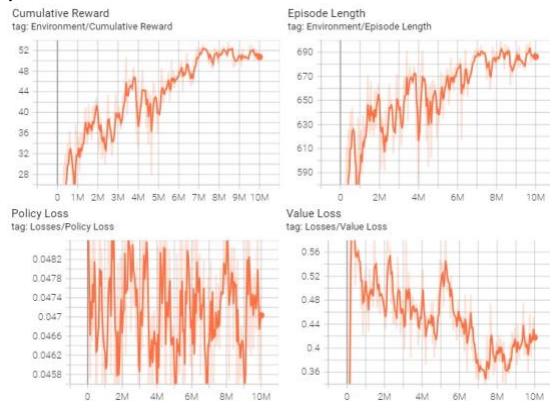
direzione da percorrere per compiere un tragitto ottimale.

Avremo dunque la seguente lista di osservazioni:

- **Velocità**
- **RPS 360sensor**;
- **RPS 360sensor2**;
- **RPS frontsensor1**;
- **RPS frontsensor2**;
- **RPS frontsensor3**;
- **RPS checksensor1**, cerca il 1° prossimo checkpoint;
- **RPS checksensor2** cerca il 2° prossimo checkpoint;
- **RPS checksensor3** cerca il 3° prossimo checkpoint;
- **CheckAngle1** angolo tra macchina e 1° prossimo checkpoint;
- **CheckAngle2** angolo tra macchina e 2° prossimo checkpoint
- **CheckAngle3** angolo tra macchina e 3° prossimo checkpoint

Tensor size : 142

A questo punto l'agente è in grado di svolgere in autonomia il percorso, nel minor tempo possibile, completando il percorso il 99% delle volte.



4.3.4 Fase 4: ottimizzazione

Il nostro compito finale è quello di utilizzare il minor numero di osservazioni per ottimizzare il processo di training.

1. Inizieremo rimuovendo alcuni RPS valutando quali, anche se rimossi, non variano le prestazioni dell'agente.

2. Rimuoveremo l'osservazione velocità, inserendo invece uno stack sull'RPS della posizione, l'agente potrà così capire la sua velocità in base alla differenza tra coordinate.

Avremo dunque la seguente lista di osservazioni:

- **RPS 360sensor2** con stack=2
- **RPS frontsensor1**;
- **RPS frontsensor2**;
- **RPS checksensor1**, cerca il 1° prossimo checkpoint;
- **RPS checksensor2** cerca il 2° prossimo checkpoint;
- **RPS checksensor3** cerca il 3° prossimo checkpoint;
- **CheckAngle1** angolo tra macchina e 1° prossimo checkpoint;
- **CheckAngle2** angolo tra macchina e 2° prossimo checkpoint
- **CheckAngle3** angolo tra macchina e 3° prossimo checkpoint

Tensor size : 139

Abbiamo dunque rimosso la velocità e il sensore 360sensor1 (il più vicino).

Alla rimozione della velocità e del sensore 360, vi è stata una variazione di performance, circa il 70% dei percorsi venivano completati.

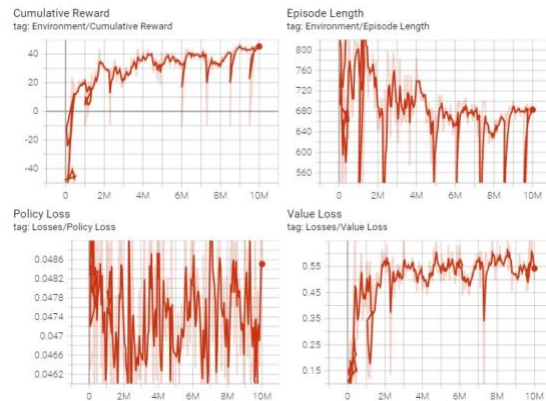
Per ridurre le dimensioni del modello sono stati effettuati vari tentativi.

La rimozione del *frontsensor3*, pur essendo il sensore meno caratterizzante tra gli RPS restanti, ha portato l'agente a completare circa il 50% dei tracciati.

Questo dimostra quanto gli RPS inseriti nella FASE3 fossero essenziali, e come la loro rimozione, possa portare ad una diminuzione di prestazioni da parte del modello.

Aumentando la dimensione dello stack di 360sensor2, otteniamo miglioramenti quasi impercettibili; pertanto, mantenere la dimensione a 2 è stata la scelta ottimale.

Il seguente grafico è stato generato dall'addestramento dell'agente della FASE3, a cui è stata rimossa l'osservazione velocità, e con uno stack di dimensione 2 sul RPS 360sensor2.



5. Conclusione

Il progetto ha dimostrato l'efficacia del reinforcement learning nella navigazione autonoma di veicoli mediante un simulatore sviluppato con Unity e ML-Agents, focalizzandosi su aspetti chiave come il riconoscimento del percorso e la gestione della velocità.

L'addestramento dell'agente attraverso un sistema di rewards e punizioni ha evidenziato l'importanza di selezionare accuratamente le osservazioni (posizione, velocità, sensori ambientali) per migliorarne le prestazioni.

L'introduzione di checkpoint e percorsi dinamici ha ulteriormente sfidato l'agente, sottolineando la necessità di un ambiente di addestramento ricco e variabile. Questo studio ha confermato che un ambiente ben configurato, unito a un sistema di reward efficace, consente di addestrare agenti per compiti di guida autonoma, dimostrando l'importanza del reinforcement learning nello sviluppo di veicoli autonomi.